Module 4: Intent Detection and Entity Resolution

How a system works out what kind of answer the user wants, and which specific things they are talking about

Learning objectives

By the end of this module, you should be able to:

1. Classify a question's **intent** with Bayesian and softmax models, including questions with more than one intent.
2. Map an intent to a **structural template** with slots to fill.
3. Choose the right **aggregation operator** from the user's words, and avoid the average-of-averages trap.
4. Resolve **relative time expressions** into exact intervals.
5. Detect **mentions** in a question and tag them by type.
6. **Score and link** each mention to a specific entity, and resolve several mentions **jointly**.
7. Decide when to **abstain** (NIL) and check whether confidence is **calibrated**.

4.1 The problem: two questions inside every question

Consider: *"What were the top 5 Paris stores by revenue last quarter?"*

Before any data is touched, two things must be decided:

Question	Answer for the example	Method
What kind of answer? (intent)	A *ranking*: top-N items ordered by a measure	Intent classification (4.2–4.5)
About which things? (entities)	Paris = which Paris? Last quarter = which dates?	Mention detection and entity linking (4.6–4.10)

Modules 1–3 found **where** the data lives. This module decides **what shape** the answer takes and **which rows** it covers.

4.2 The intent space

Define a finite set of intents:

```
K = { lookup, count, aggregate, trend, compare, rank, distribution, explain }
```

Intent	Typical phrasing	Answer shape
lookup	"What is X for Y?"	A single value
count	"How many…?"	A single integer
aggregate	"Total / average X by Y"	Value per group
trend	"How has X changed over time?"	Time series

compare	"X vs Y", "compared to last year"	Paired values and a difference
rank	"Top / bottom N", "best", "worst"	Ordered list
distribution	"Spread / breakdown of X"	Histogram or shares
explain	"Why did X drop?"	Decomposition of a change

Single-label vs. multi-label

Some questions carry **more than one** intent. *"Why did revenue drop last month compared to the month before?"* is both **explain** and **compare**.

- **Single-label:** pick one intent, so probabilities sum to 1. Use softmax.
- **Multi-label:** each intent is an independent yes/no, so probabilities don't sum to 1. Use one sigmoid per intent.

```
Single:  P(k | Q) = softmax_k( W_k · q + b_k )
Multi:   P(k ∈ Q) = σ( W_k · q + b_k )       include intent k if P > 0.5
```

4.3 Classifying intent: a Bayesian view

The simplest interpretable classifier is **naive Bayes over cue words**:

```
P(k | Q) ∝ P(k) · Π_{t ∈ Q} P(t | k)
```

The "naive" assumption is that cue words are independent given the intent. That is false in general, but often good enough to rank intents correctly.

Worked example

Question: *"top 5 regions by revenue"*. The cue words are **"top"** and **"by"**.

Intent k	Prior P(k)	P("top" | k)	P("by" | k)	Product
rank	0.30	0.30	0.20	0.30 × 0.30 × 0.20 = **0.0180**
trend	0.30	0.02	0.15	0.30 × 0.02 × 0.15 = **0.0009**
lookup	0.40	0.03	0.10	0.40 × 0.03 × 0.10 = **0.0012**

The normalizer is 0.0180 + 0.0009 + 0.0012 = **0.0201**.

```
P(rank | Q)   = 0.0180 / 0.0201 ≈ 0.896
P(trend | Q)  = 0.0009 / 0.0201 ≈ 0.045
P(lookup | Q) = 0.0012 / 0.0201 ≈ 0.060
```

Rank wins decisively, driven mostly by "top". The word "by" appears in many intents, so it carries little information.

> **Information per cue word.** A word's usefulness is how much it shifts the odds between intents: log [$P(t \mid k_1) / P(t \mid k_2)$]. For "top", rank vs. trend gives log(0.30 / 0.02) ≈ 2.7 nats, a strong cue. For "by", it gives log(0.20 / 0.15) ≈ 0.29 nats, a weak one.

Beyond word counts

Modern classifiers replace the cue-word table with an embedding $q = \varphi(Q)$, so paraphrases such as "highest-performing", "leading" and "best" land near "top" automatically. The Bayesian structure, a prior times evidence, stays the same.

4.4 From intent to template

Each intent implies a **template**: a skeleton with slots that the rest of the pipeline must fill.

```
template(rank)    = ⟨ measure, dimension, order direction, N, filters ⟩
template(trend)   = ⟨ measure, time grain, time range, filters ⟩
template(compare) = ⟨ measure, side A, side B, difference type ⟩
template(explain) = ⟨ measure, period₀, period₁, candidate drivers ⟩
```

Example: *"top 5 Paris stores by revenue last quarter"*

Slot	Filled value	Source
measure	Revenue	Concept mapping (Module 2)
dimension	Store	Mention "stores"
order direction	Descending	"top"
N	5	Number in the question
filter: location	Paris → *which one?*	Entity linking (4.8)
filter: time	Last quarter → *which dates?*	Time resolution (4.6)

Slot completeness

```
complete(template) = 1  ⟺  every required slot is filled with confidence ≥ τ
```

When a required slot is empty, fill it with a prior if it's low-risk (Module 3, section 3.9), or ask the user.

The "explain" template is a decomposition

For "why did X change?", the answer splits the total change into contributions from each segment s:

$$\Delta X = X_1 - X_0 = \Sigma_s (X_{1,s} - X_{0,s}) = \Sigma_s \Delta_s$$

$$\text{share}_s = \Delta_s / \Delta X \qquad \leftarrow \text{fraction of the change caused by segment } s$$

Example: revenue fell by $100k. The North region accounts for -$80k, and the rest is spread thinly, so North's share is **80%** of the drop.

4.5 Choosing the aggregation operator

The operator comes from the **modifier words in the question**, not from the column name. A column called `avg_price` still needs SUM when the user asks for "total".

```
op* = argmax_{op}  P(op | modifier words in Q)
```

Modifier cue	Operator
"total", "overall", "sum of"	SUM
"average", "mean", "typical", "per"	AVG (weighted, see below)
"highest", "largest", "peak", "maximum"	MAX
"lowest", "smallest", "minimum"	MIN
"how many [entities]"	COUNT DISTINCT entity
"how many [records / transactions]"	COUNT rows
"median", "middle"	PERCENTILE 0.5

Pitfall 1: rows vs. entities

"How many customers placed orders?" There are 10 order rows from 4 distinct customers.

```
COUNT(rows)                = 10   ✗  (counts orders)
COUNT(DISTINCT customer)   = 4    ✓  (counts customers)
```

The **noun** after "how many" decides which entity is counted.

Pitfall 2: average of averages

Region	Orders	Average order value
A	10	$100
B	90	$200

```
Naive:     (100 + 200) / 2                    = $150   ✗
Weighted:  (10 × 100 + 90 × 200) / (10 + 90)
         = (1,000 + 18,000) / 100             = $190   ✓
```

In general:

```
true average = Σ_g n_g · avg_g / Σ_g n_g       (weight each group by its size)
```

> **Rule:** averages, ratios and percentages are **not additive**. Re-compute them from their parts, Σ numerator / Σ denominator, never by averaging pre-computed averages.

4.6 Resolving time expressions

Relative phrases need an **anchor** date (today) and a **calendar convention** to become exact intervals. We use half-open intervals [start, end), so adjacent periods never overlap or leave gaps.

Suppose today is **2026-10-21** and the calendar is a standard calendar year:

Phrase	Interval
"last quarter"	[2026-07-01, 2026-10-01): Q3 2026
"last month"	[2026-09-01, 2026-10-01)
"year to date"	[2026-01-01, 2026-10-22)
"last 30 days"	[2026-09-21, 2026-10-21), or up to 10-22 if today counts
"same quarter last year"	[2025-07-01, 2025-10-01)

Ambiguity is real

- **Fiscal vs. calendar:** if the fiscal year starts on 1 February, "last quarter" is a different interval.
- **Complete vs. partial:** does "this month" include today's incomplete data?
- **Rolling vs. calendar:** "last 3 months" can mean the previous three full months or the trailing 90 days.

These are latent variables, as in Module 3:

```
interval* = argmax_I  P(I | phrase, convention, intent)
```

A **trend** intent prefers complete periods, so the last partial month doesn't create a fake drop. A **lookup** of "this month so far" accepts a partial period.

> **Fake-drop trap:** comparing a partial current period with a full previous one makes almost every metric look like it fell. A month that is 21 of 31 days complete shows about a 32% "drop" even at a perfectly steady rate (1 - 21/31 ≈ 0.32).

4.7 Detecting mentions

A **mention** is a span of words that refers to something resolvable: a metric, an entity value, an entity type, a time or a number.

Example: *"top 5 **Paris stores** by **revenue last quarter**"*

Span	Type
"5"	NUMBER (N for top-N)

"Paris"	ENTITY VALUE (location)
"stores"	ENTITY TYPE
"revenue"	METRIC
"last quarter"	TIME

Sequence tagging (BIO scheme)

Each token gets a tag: **B**-type (beginning of a span), **I**-type (inside a span) or **O** (outside any span).

```
top   5      Paris   stores   by   revenue    last     quarter
O     B-NUM  B-LOC   B-TYPE   O    B-METRIC   B-TIME   I-TIME
```

The most likely tag sequence y* is the one with the best total score, found with the Viterbi algorithm:

$$y^* = \text{argmax}_y \ \Sigma_i \ [\ \text{emit}(x_i, y_i) + \text{trans}(y_{i-1}, y_i)\ ]$$

The transition scores enforce consistency. For example, I-TIME can only follow B-TIME or I-TIME, never O.

4.8 Linking mentions to entities

Each detected mention m must resolve to a concrete entity e from a set of candidates E(m).

Step 1: Generate candidates

Gather plausible entities using the lexical and semantic matching from Module 2: exact matches, near spellings and known aliases. Keep the top few ($|E(m)| \approx 5–20$).

Step 2: Score each candidate

$$\begin{aligned} \text{score}(m, e) = {} & \theta_1 \cdot \text{sim_lex}(m, e) && \leftarrow \text{spelling match} \\ & + \theta_2 \cdot P(e \mid m) && \leftarrow \text{prior: how often } m \text{ refers to } e \\ & + \theta_3 \cdot \text{type_fit}(e, \text{slot}) && \leftarrow \text{does } e \text{ have the type the template expects?} \\ & + \theta_4 \cdot \text{coherence}(e, Q) && \leftarrow \text{fit with the other concepts in the question} \end{aligned}$$

Worked example: which "Paris"?

Question: *"top 5 **Paris** stores by revenue"*. The template expects a **location**, because the word "stores" signals a place filter.

Weights: $\theta = (1, 1, 2, 1)$. Type fit counts double.

Candidate e	sim_lex	P(e | m)	type_fit	coherence	score
Paris, France (city)	1.0	0.6	1.0	0.8	1.0 + 0.6 + 2.0 + 0.8 = **4.4**
Paris, Texas (city)	1.0	0.1	1.0	0.3	1.0 + 0.1 + 2.0 + 0.3 = **3.4**

Paris Retail Ltd (customer)	0.7	0.2	0.2	0.2	0.7 + 0.2 + 0.4 + 0.2 = **1.5**
"Paris" perfume line (product)	0.6	0.1	0.0	0.1	0.6 + 0.1 + 0.0 + 0.1 = **0.8**

Softmax (T = 1):

```
e^4.4 ≈ 81.45    e^3.4 ≈ 29.96    e^1.5 ≈ 4.48    e^0.8 ≈ 2.23     Σ ≈ 118.12

P(Paris, France) ≈ 0.690
P(Paris, Texas)  ≈ 0.254
P(Paris Retail)  ≈ 0.038
P(Paris perfume) ≈ 0.019
```

Decision rule ($\tau = 0.6$, $\delta = 0.2$ from Module 2): $P = 0.69 \geq 0.6$, and the margin $0.69 - 0.25 = 0.44 \geq 0.2$, so **accept Paris, France**.

Step 3: Check against the data

A link is only useful if the entity actually appears in the relevant data. Update with the evidence that stores exist there:

$$P(e \mid m, \text{data}) \propto P(e \mid m) \cdot P(\text{stores exist at } e)$$

If the store data held 12 stores in Paris, France and 0 in Paris, Texas, the Texas candidate would drop to about zero. If the reverse were true, the decision would **flip**. The data has the final say.

4.9 Resolving several mentions jointly

When a question has several ambiguous mentions, resolving them one at a time can produce **incoherent combinations**. Resolve them **together**:

$$(\hat{e}_1, \ldots, \hat{e}_j) = \operatorname{argmax}_{e_1 \ldots e_j} \; \Sigma_i \, \text{score}(m_i, e_i) \; + \; \Sigma_{i<l} \, \psi(e_i, e_l)$$

Here $\psi(e_i, e_l)$ is a **pairwise compatibility** bonus. For example, two cities in the same country or state are more likely to be mentioned together.

Worked example: "Compare Paris and Austin stores"

"Austin" resolves strongly to Austin, Texas; call its score s_A. For Paris, the individual scores are the same as in 4.8.

Compatibility: ψ(Paris TX, Austin TX) = **+1.5** (same state), and ψ(Paris FR, Austin TX) = **0**.

Joint assignment	Individual scores	Compatibility	Total
(Paris FR, Austin TX)	4.4 + s_A	0	**4.4 + s_A**
(Paris TX, Austin TX)	3.4 + s_A	1.5	**4.9 + s_A**

The joint decision flips to Paris, Texas. Comparing two Texas cities is more coherent.

How confident is it? Softmax over the two leading assignments (s_A cancels out):

```
P(Paris TX, Austin TX) = 1 / (1 + e^(−0.5)) ≈ 0.62
P(Paris FR, Austin TX) ≈ 0.38
```

The result is 0.62 ≥ 0.6 with a margin of 0.24 ≥ 0.2, so it is **accepted, but only just**. A borderline result like this should be **stated as an assumption** ("interpreting Paris as Paris, Texas").

Computational cost

With j mentions and K candidates each, there are K^j joint assignments. Ten mentions with 10 candidates each gives 10^{10} combinations, far too many to enumerate. Practical approaches:

Method	Idea
Beam search	Keep only the top-B partial assignments at each step
Loopy belief propagation	Pass "messages" between mentions until their beliefs stabilize
Coordinate ascent	Fix all mentions but one, improve it, then repeat

4.10 Abstaining: the NIL option

Sometimes **no candidate is right**: the entity is new, misspelled beyond recognition, or simply not in the data. Forcing a match is worse than admitting uncertainty.

```
ê(m) = argmax_e score(m, e)    if  max_e score(m, e) ≥ τ_NIL
       NIL                     otherwise   →  ask, or report "not found"
```

An equivalent approach adds NIL as an explicit candidate with a fixed score s_NIL. It wins whenever every real candidate scores below it.

Calibration: does 70% confidence mean 70% correct?

A system is **calibrated** if, among all answers given with confidence p, a fraction of about p are actually correct. Measure this with the **expected calibration error**: group predictions into confidence bins $B_1 \ldots B_M$, then

```
ECE = Σ_m ( |B_m| / n ) · | accuracy(B_m) − confidence(B_m) |
```

Example: 100 links made at about 90% confidence, of which only 70 were correct. That bin contributes |0.70 - 0.90| = 0.20 of error, weighted by its share of all links. The system is **overconfident**.

Fixing miscalibration: temperature scaling

Divide all scores by a learned temperature T before the softmax:

```
P(e | m) = softmax( score / T )
T > 1  →  softer probabilities (fixes overconfidence)
T < 1  →  sharper probabilities (fixes underconfidence)
```

The ranking doesn't change, only the stated confidence, so the thresholds τ and δ become trustworthy.

4.11 Putting it together: the parse score

A full **parse** of the question combines the intent, template slots, operators, time intervals and entity links:

```
log P(parse | Q) =   log P(k̂ | Q)                                ← intent            (4.2–4.3)
                   + Σ_slots log P(slot filled correctly)        ← template          (4.4)
                   + log P(op* | modifiers)                      ← aggregation       (4.5)
                   + log P(interval* | phrase, convention)       ← time              (4.6)
                   + Σᵢ log P(êᵢ | mᵢ)                           ← entity links      (4.8)
                   + Σ_{i<l} ψ(êᵢ, ê_l)                          ← joint coherence   (4.9)
```

The best parse is the argmax of this score. Each component with low confidence becomes a **stated assumption** or a **clarifying question**.

End-to-end example

"top 5 Paris stores by revenue last quarter" →

Component	Resolution	Confidence
Intent	rank	0.90
Measure / operator	Revenue / SUM	High
Dimension	Store	High
Order, N	Descending, 5	High
Location	Paris, France	0.69
Time	[2026-07-01, 2026-10-01)	High (calendar convention assumed)

4.12 Common pitfalls

Pitfall	Symptom	Fix in the math
Forcing a single intent	"Why… compared to…" loses the comparison	Multi-label sigmoids
Operator chosen from the column name	"Total" answered with an average	op* from the modifier words in Q
COUNT rows vs. COUNT DISTINCT	Counting orders when asked for customers	The noun after "how many" picks the entity
Average of averages	Wrong overall mean	Σ numerator / Σ denominator

Partial-period comparison	Fake drops in the current period	Prefer complete periods for trends
Resolving mentions independently	Incoherent pairs (Paris, FR + Austin, TX)	Joint assignment with ψ
Forced linking	Wrong entity instead of "not found"	NIL threshold τ_NIL
Overconfident scores	Thresholds fire wrongly	Calibrate with temperature scaling

Check your understanding

1. **Compute:** equal priors (0.5) for trend and rank. P("over time" | trend) = 0.40, and P("over time" | rank) = 0.05. What is P(trend | Q)?
2. **Compute:** region A has 20 orders averaging $50, and region B has 80 orders averaging $100. What are the naive and correct overall averages?
3. **Compute:** 500 orders came from 120 distinct customers. *"How many customers bought?"*
4. **Compute:** two candidate entities score 3 and 1 (T = 1). What is the probability of the top one?
5. **Reason:** why can resolving mentions jointly change an answer that looked confident in isolation?
6. **Apply:** today is 2026-10-21. What interval is "last full month"?

<details>
<summary>Answers</summary>

1. 0.5 × 0.40 = 0.20 and 0.5 × 0.05 = 0.025. P(trend) = 0.20 / 0.225 ≈ **0.889**.
2. Naive: (50 + 100) / 2 = **$75**. Correct: (20 × 50 + 80 × 100) / 100 = (1,000 + 8,000) / 100 = **$90**.
3. **120**, using COUNT DISTINCT customer, not 500.
4. 1 / (1 + e^(−2)) ≈ **0.881**.
5. Pairwise compatibility ψ adds evidence that individual scoring can't see. A candidate that is second-best alone can win when it fits much better with the *other* entities in the question.
6. **[2026-09-01, 2026-10-01)**, all of September 2026.

</details>

Key takeaways

- **Intent** sets the answer's shape. Use softmax for one intent, sigmoids for several, and cue-word likelihood ratios to show which words matter.
- Each intent maps to a **template with slots**. Explain-type intents become **decompositions** of a change.
- The **aggregation operator comes from the user's words**. Count the right entity, and never average averages.
- **Time phrases** need an anchor date and a calendar convention. Prefer complete periods for trends.
- **Entity linking** = candidate generation + scoring (lexical, prior, type, context) + checking against the data.
- **Joint resolution** with pairwise compatibility catches incoherent combinations, but borderline wins should be stated as assumptions.
- **NIL and calibration** keep the system honest: "not found" beats a confident wrong match.

Coming next: Module 5, Semantic Matching, Confidence and Ranking

Now that the question is parsed, which **specific fields** answer each slot? We'll cover bipartite matching (the Hungarian algorithm), value validation and fuzzy lookup, combining stage-by-stage confidence, sanity checks and anomaly bounds, two-stage retrieval and re-ranking, learning to rank, diversity (MMR), and evaluating rankings with nDCG.

Report generated by Databricks Genie